Budget Bites: Smart Recipe Planner with AI

Samara Pires (1864538), Saung Hnin Phyu (1931834), Anna Tam Ly (1915394)

1 Introduction **2**
2 Literature Review **2**
3 Methodology **3**
3.1 System Overview 3
3.2 Data Collection and Cleaning 4
3.3 Recipe Representation and FAISS Retrieval 4
3.4 Retrieval-Augmented LLM Generation 5
3.5 Streamlit User Interface 6
3.6 Baseline System for Comparison 6
3.7 End-to-End Workflow 6
4 Experiment **6**
4.1 Experimental Setting 6
4.2 Datasets 7
4.3 Experiment Design 7
4.4 Experimental Observations 8
4.5 System Demonstration (RAG-Based Chatbot) 9
5 Evaluation **9**
5.1 Baselines 9
5.2 Evaluation Metrics 9
5.3 Market Survey 10
5.4 Evaluation Result 11
5.5 Demonstration of Baseline vs RAG Comparison 12
5.6 Analysis 12
6 Conclusion **13**
6.1 Main Findings 13
6.2 Contributions 13
6.3 Limitations 13
6.4 Future Work 14
7 Individual Contribution Statement **14**
8 Appendix **14**
9 References **15**

1 Introduction

The transition to independent living, particularly among college and university students, often brings significant challenges in maintaining healthy eating habits. Due to demanding academic schedules, time constraints, and limited resources, many students tend to rely heavily on fast food and processed meals. While convenient, these choices frequently lack nutritional balance and can contribute to long-term health concerns, including poor diet quality and increased risk of lifestyle-related conditions.

At the same time, a large proportion of students possess basic cooking ingredients but lack the time, planning, or guidance required to convert them into simple, healthy meals. This gap between available resources and actionable meal preparation highlights an opportunity for intelligent systems that can support quick, practical, and budget-conscious cooking decisions.

Recent advancements in artificial intelligence - particularly Large Language Models (LLMs) - have enabled the development of conversational systems capable of generating personalized and context-aware recommendations. In the domain of food and cooking, these systems can move beyond static recipe databases and instead provide dynamic, user-specific meal suggestions through natural language interaction.

Motivated by these challenges, this project introduces **Budget Bites**, an AI-powered recipe chatbot designed to assist users - especially college students - in preparing affordable, quick, and healthier meals. The system takes into account constraints such as available ingredients, time limitations, and budget considerations, and generates tailored recipe suggestions through an interactive conversational interface.

By combining accessibility, personalization, and practicality, Budget Bites aims to reduce dependence on unhealthy food choices, promote better eating habits, and support efficient meal planning for individuals with busy lifestyles.

While the system is motivated by the challenges faced by college students, Budget Bites is designed to be applicable to a much broader audience. It can support any individual who is aware of the ingredients they have on hand but requires guidance in transforming them into meaningful meals, while also considering basic nutritional aspects such as macronutrient balance. By combining simplicity with personalized recommendations, the overarching goal of Budget Bites is to encourage better everyday food choices - helping users eat a little better, a little healthier, and more consciously.

2 Literature Review

The development of intelligent recipe recommendation systems has progressed from simple keyword-based matching toward more advanced approaches that combine semantic understanding, retrieval techniques, and generative models. Traditional systems primarily relied on content-based filtering and ingredient matching, where recipes were retrieved based on overlap between user-provided ingredients and stored recipe data. While effective in basic scenarios, these methods struggle to handle complex user constraints such as cooking time, nutritional goals, and budget considerations, and lack the flexibility required for natural language interaction [1].

To overcome these limitations, recent research has focused on incorporating **natural language processing and Large Language Models (LLMs)** into recommendation systems. LLMs enable systems to interpret open-ended user inputs and generate context-aware responses, making them particularly suitable for conversational applications such as recipe assistants. Unlike rule-based chatbots, LLMs can reason over multiple constraints simultaneously and produce structured outputs. However, LLMs alone are prone to issues such as hallucination and lack of grounding in specific datasets, which can reduce the reliability of generated responses [2].

To address these challenges, **Retrieval-Augmented Generation (RAG)** has emerged as a powerful framework that combines retrieval-based and generative approaches. In RAG systems, relevant information is first retrieved from an external knowledge base and then used as context for generation. Lewis et al. demonstrated that this hybrid approach improves factual accuracy and relevance by

integrating parametric knowledge from pre-trained models with non-parametric knowledge stored in external databases [3]. This architecture is particularly well-suited for applications such as recipe recommendation, where both accurate retrieval and flexible response generation are required.

A critical component of RAG systems is the use of **text embeddings**, which convert textual data into dense numerical vector representations. These embeddings capture semantic meaning, enabling similarity comparisons beyond exact keyword matching. Sentence-BERT, proposed by Reimers and Gurevych, introduced an efficient method for generating sentence-level embeddings that can be compared using cosine similarity [4]. In recipe recommendation systems, embeddings allow user queries such as "quick high-protein meal" to be matched with relevant recipes even when the wording differs significantly.

To efficiently perform similarity search over large embedding datasets, modern systems employ specialized vector search libraries such as **FAISS (Facebook AI Similarity Search)**. FAISS enables fast nearest-neighbor search in high-dimensional vector spaces, making it possible to retrieve the most relevant items from large datasets in real time. Johnson et al. demonstrated that FAISS can scale to billion-level datasets while maintaining high efficiency [5]. In the context of recipe recommendation, FAISS supports semantic retrieval of recipes based on user queries, forming the retrieval backbone of the system.

In addition to semantic retrieval, practical systems must also incorporate **constraint-based filtering and ranking mechanisms**. While embedding-based retrieval identifies semantically similar candidates, additional filtering ensures that retrieved results satisfy user-defined constraints such as cooking time, cuisine, or dietary goals. Ranking strategies, such as sorting by nutritional attributes (e.g., protein, calories, fat), further refine the results. This combination of semantic retrieval and rule-based filtering enables systems to balance flexibility with precision in real-world applications.

Another important aspect of LLM-based systems is **prompt engineering**, which involves designing structured instructions to guide model outputs. Effective prompts ensure that generated responses are consistent, interpretable, and aligned with user constraints. Research and industry practices highlight that well-designed prompts can significantly improve the reliability and usability of LLM outputs, particularly in structured tasks such as recipe generation.

Finally, the usability of such systems depends on accessible and interactive user interfaces. Tools like **Streamlit** have become widely adopted for building lightweight, interactive machine learning applications. Streamlit enables rapid deployment of applications where users can input constraints and receive structured outputs in real time, making it well-suited for conversational recommendation systems [6].

Overall, existing literature suggests that effective AI-powered recommendation systems should integrate multiple components: semantic embeddings for understanding user queries, vector search for efficient retrieval, retrieval-augmented generation for grounded responses, and structured prompting for consistent output. The methodology adopted in Budget Bites builds directly on these foundations by combining FAISS-based semantic retrieval, constraint-based filtering, and LLM-based generation within an interactive Streamlit application, enabling practical and personalized recipe recommendations.

3 Methodology

3.1 System Overview

Budget Bites was implemented as a retrieval-augmented recipe recommendation system. The system takes user constraints, retrieves relevant recipes from a cleaned recipe dataset, and uses a large language model to generate one final structured recipe recommendation.

The overall workflow begins when the user enters their available ingredients, maximum cooking time, cuisine preference, nutrition goal, and recipe category through the Streamlit interface. These inputs are converted into a structured search query. The query is embedded using a sentence-transformer model and compared with recipe embeddings stored in a FAISS index. The most relevant recipes are retrieved

from the dataset and filtered according to the user's constraints. The retrieved recipes and original user inputs are then passed to the LLM, which generates the final recipe output.

The system output is displayed in a fixed format: recipe name, description, ingredients, steps, total time, calories, protein, fats, and carbohydrates. This structured format was used so the Streamlit app could parse the LLM response and present each part of the recipe clearly.

3.2 Data Collection and Cleaning

The recipe knowledge base was built using the Recipe Dataset from Kaggle. This dataset was selected because it contains the main recipe-level information needed for Budget Bites, including recipe name, category, cuisine, preparation time, total time, serving information, nutrition, description, recipe steps, and ingredients. These fields directly match the app requirements because the system needs to retrieve recipes based on ingredients, maximum cooking time, cuisine, nutrition goal, and category, then return a structured recipe recommendation.

The original dataset contained 486 recipes and 12 columns. The cleaned dataset contained 483 recipes and 20 columns. During preprocessing, the dataset was cleaned, completed, and reformatted so that it could be used reliably in the retrieval system and Streamlit app.

A major part of the preprocessing step was completing important recipe fields that were empty, incomplete, or not usable for the final app. In particular, the columns Nutrition (per serving), Description, Recipe_steps, and Ingredients were reviewed and filled where needed. This was necessary because these columns are central to the system. The retrieval model uses ingredients, descriptions, cuisine, category, time, and nutrition information to find relevant recipes, while the final app displays the ingredients, steps, and nutrition values directly to the user.

For missing or incomplete Description values, short recipe summaries were added so each recipe had a clear explanation of the dish. For incomplete Ingredients values, usable ingredient lists were written in comma-separated form. For missing or weak Recipe_steps, clear numbered cooking instructions were added so the final output would be practical for users. For Nutrition (per serving), values were written in a consistent format containing calories, protein, fats, and carbohydrates.

After completing the missing content, the dataset was standardized. The original column name Decription was corrected to Description, and Nutrition (per serving) was renamed to Nutrition_per_serving for easier coding. Text fields were cleaned by removing unnecessary spacing, inconsistent formatting, brackets, quotation marks, and list-style formatting. Cuisine and category values were also standardized so that user-selected filters in the app would match the dataset values correctly.

Time values were converted into numeric minutes because the app uses maximum cooking time as one of the user constraints. Rows with invalid or inconsistent time values were reviewed. In particular, recipes where total time was smaller than preparation time were flagged because total time should not be less than preparation time.

Nutrition information was extracted into separate numeric columns: Calories, Protein_g, Fats_g, and Carbs_g. These new columns were needed because the app supports nutrition goals such as High protein, Low calorie, and Low fat. Instead of treating nutrition as one block of text, the cleaned dataset allows the system to sort or filter recipes using specific nutrition values.

Additional flag columns were also created to identify possible issues in the dataset: Flag_bad_time, Flag_bad_ingredients, Flag_bad_steps, and Flag_bad_nutrition. These flags helped check whether recipes had problems in important fields before being used in the final system. After cleaning, completing, and reviewing the dataset, duplicate or unusable rows were removed, resulting in the final cleaned dataset used for retrieval and recipe generation.

3.3 Recipe Representation and FAISS Retrieval

After the cleaned dataset was finalized, a retrieval-ready version of the dataset was created. This version included an additional column called recipe_text, which combined the most important recipe information into one searchable text field. Each recipe text included the recipe name, category, cuisine, ingredients, description, total time, calories, protein, fats, and carbohydrates.

This representation was used because user requests are usually not based on recipe titles alone. A user may enter ingredients such as “chicken, rice, onion” while also requesting a high-protein meal or a specific cuisine. By combining ingredients, cuisine, category, description, time, and nutrition values into one text field, the retrieval system can search based on both recipe content and user constraints.

Each recipe_text entry was converted into a numerical embedding using the all-MiniLM-L6-v2 model from Sentence Transformers. The same model was used later to embed the user query. The recipe embeddings were converted into float32 format and stored in a FAISS index. FAISS was used to perform fast vector similarity search over the recipe database.

When a user submits a request, the app constructs a query using the available ingredients, cuisine preference, nutrition goal, and category. The query is then embedded and searched against the FAISS index. The system first retrieves a larger set of candidate recipes and then applies rule-based filtering. Recipes are filtered by maximum cooking time, cuisine, and category when those constraints are provided. Nutrition goals are handled by ranking: high-protein requests are sorted by protein, low-calorie requests are sorted by calories, and low-fat requests are sorted by fats.

This retrieval design combines semantic search with deterministic constraint filtering. FAISS helps identify recipes that are semantically similar to the user request, while filtering ensures that practical constraints such as cooking time, cuisine, and category are respected.

3.4 Retrieval-Augmented LLM Generation

The full Budget Bites system uses retrieval-augmented generation. Instead of asking the LLM to generate a recipe from the user input alone, the system first retrieves relevant recipes from the cleaned dataset. These retrieved recipes are then passed into the LLM as grounding context.

Each retrieved recipe is formatted into a prompt block containing the recipe name, category, cuisine, description, ingredients, recipe steps, total time, calories, protein, fats, and carbohydrates. This gives the LLM concrete examples from the dataset before it generates the final recommendation.

The final prompt gives the LLM two sources of information: the original user constraints and the retrieved recipes. The prompt instructs the LLM to generate one final recipe recommendation based on both. It also gives specific rules: the recipe should be practical and realistic, match the user’s available ingredients as closely as possible, respect the maximum cooking time if provided, respect cuisine/category/nutrition constraints, and use the retrieved recipes as guidance.

The LLM response is required to follow a fixed format:

- recipe name,
- description,
- ingredients,
- steps,
- total time,
- calories,
- protein,
- fats,
- carbohydrates.

The implementation uses Gemini 2.5 Flash through the Google Generative AI API. In this design, the LLM’s role is to synthesize the final user-facing recipe from retrieved examples, rather than acting as the only source of recipe knowledge.

The system also includes a fallback strategy. If the initial retrieval returns no matching recipes under the strict constraints, the system relaxes the constraints and searches again using the ingredients only. If retrieval still fails, the app falls back to the baseline LLM-only response. This prevents the system from crashing when the user enters unusual ingredients or very narrow constraints.

3.5 Streamlit User Interface

The user interface was built using Streamlit. The app page is titled Budget Bites and uses a centered layout with a logo, a short description, input fields, a system mode selector, and a final output section.

The user input fields include ingredients, maximum cooking time, nutrition goal, cuisine, and category. The ingredients field is a text area, maximum cooking time is a numeric input, and nutrition goal is selected from options such as Any, High protein, Low calorie, and Low fat. Cuisine and category are selected from dropdown menus created from the values available in the dataset.

The interface also allows the user to choose between two systems: Full system (RAG) and Baseline (LLM only). This makes it possible to enter the same input into both systems and compare the results directly.

After the recipe is generated, the app displays the output under separate subheadings: recipe name, description, ingredients, steps, total time, calories, protein, fats, and carbs. The recipe steps are formatted so that numbered instructions appear on separate lines. For the full RAG system, the app also displays the top retrieved recipes in a table, including recipe name, cuisine, category, total time, calories, protein, fats, and carbohydrates.

3.6 Baseline System for Comparison

To support the evaluation, we implemented a baseline LLM-only system alongside the full retrieval-augmented system. In the baseline mode, the user input is passed directly to the LLM without retrieving recipes from the FAISS index. This means the baseline does not use the cleaned recipe dataset as grounding context and instead relies only on the LLM's general recipe-generation ability.

The Streamlit interface includes a system selector that allows the same user input to be tested under both modes: Baseline (LLM only) and Full system (RAG). This implementation makes it possible to compare whether adding retrieval improves the relevance, constraint satisfaction, and practicality of the generated recipe.

3.7 End-to-End Workflow

The final Budget Bites system follows this pipeline:

1. The app loads the cleaned retrieval dataset, the sentence-transformer embedding model, and the FAISS index.
2. The user enters available ingredients and optional constraints through the Streamlit interface.
3. The app converts the user inputs into a structured query.
4. The query is embedded using the same model used for the recipe embeddings.
5. FAISS retrieves candidate recipes that are semantically similar to the query.
6. The system filters candidates by maximum cooking time, cuisine, and category.
7. The system ranks recipes by nutrition goal when applicable.
8. The top retrieved recipes are formatted into prompt-ready text.
9. The LLM receives the user constraints and retrieves recipes.
10. The LLM generates one final structured recipe recommendation.
11. The app parses the response into separate output fields.
12. Streamlit displays the final recipe and, for the RAG system, the top retrieved recipes.

4 Experiment

4.1 Experimental Setting

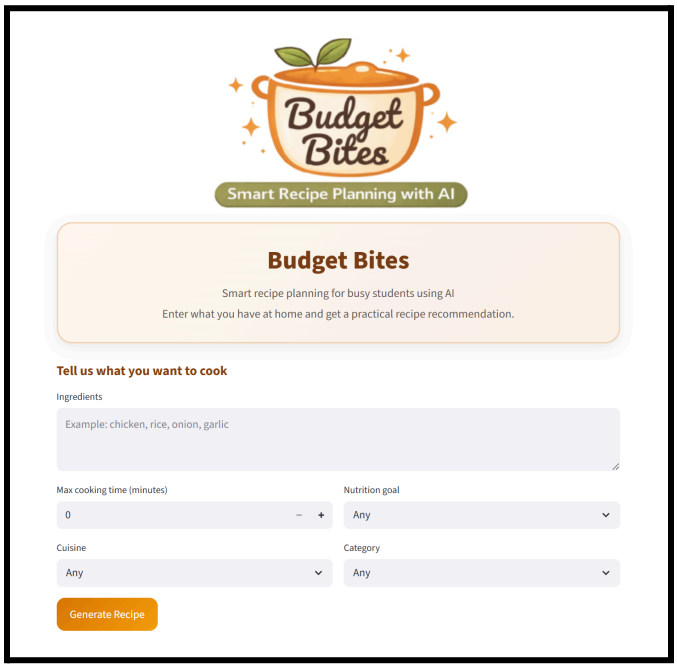
To demonstrate the functionality of the Budget Bites system, we conducted a series of experiments using the full chatbot pipeline, which integrates a Large Language Model (LLM) with Retrieval-Augmented Generation (RAG) and structured user inputs.

We implemented a recipe recommendation system that combines semantic retrieval with a large language model. Recipe data was embedded using the SentenceTransformers model (all-MiniLM-L6-v2)

and stored in a FAISS index for efficient similarity search. For generations, we used Google Gemini (gemini-2.5-flash).

The system allows users to generate recipes by specifying one or more of the following parameters:

- Ingredients
- Cooking time
- Cuisine
- Nutrition goal
- Category



System Overview

The objective of these experiments is to evaluate how the system behaves under different input combinations and levels of constraint.

4.2 Datasets

We used a recipe dataset originally obtained from Kaggle. However, the raw dataset was incomplete and lacked several important fields required for our application. To address this, we augmented the dataset by adding the following columns, Description, Recipe Steps, Ingredients and Nutrition information (calories, protein, fats, carbs). These missing fields were generated through a combination of literature-based lookup for nutritional information where possible and a large language model, OpenAI, to infer structured recipe details such as descriptions, steps, and ingredient lists.

We then performed data cleaning and preprocessing, including removing missing or inconsistent entries, standardizing text formats across columns and ensuring logical consistency, for example, total cooking time is greater than or equal to prep time. The final dataset is a cleaned and enriched version of the original Kaggle dataset, making it suitable for both retrieval and generation tasks in our system.

4.3 Experiment Design

We designed multiple test scenarios to reflect realistic user behavior, ranging from minimal input to fully specified queries. Each test case explores how the system adapts to varying levels of detail.

Test Case 1: Full Filter Recipe Generation

- Inputs: Ingredient, Cooking Time, Cuisine, Category

- Purpose: Evaluate system performance under well-defined constraints

Test Case 2: Minimal Input Scenario
- Inputs: Ingredient, Category
- Purpose: Test system flexibility with limited user input

Test Case 3: Complete Input with Nutrition Goal
- Inputs: Ingredient, Cooking Time, Cuisine, Category, Nutrition Goal
- Purpose: Evaluate system performance with all constraints applied

Test Case 4: Consistency Check
- Inputs: Ingredient, Cooking Time, Cuisine, Category
- Purpose: Assess consistency and stability of generated outputs

Test Case 5: Health-Focused Scenario
- Inputs: Ingredient, Nutrition Goal, Category
- Purpose: Evaluate ability to prioritize nutritional constraints

4.4 Experimental Observations

Test Case	Input Group	Input Constraints	Output Recipe	Performance Summary
Test Case 1: Full Filter Recipe Generation	Ingredient, Cooking Time, Cuisine, Category	Banana, Butter, 15 min, European, Dessert	Banana Flambe	The system performs well under full constraints, generating a relevant and time-compliant recipe.
Test Case 2: Minimal Input Scenario	Ingredient, Category	Apple, Cinnamon, Snack	Student-Friendly Apple Cinnamon Muffins	The system generates a coherent recipe but fails to infer reasonable constraints, producing a time-intensive output. This highlights weak adaptability when inputs are minimal.
Test Case 3: Complete Input with Nutrition Goal	Ingredient, Cooking Time, Cuisine, Category, Nutrition Goal	Chicken, Onion, Tomatoes, 70 min, Indian, Lunch, High Protein	Spicy Chicken Curry	The system performs strongly when all constraints are provided, generating a recipe that aligns well with cuisine, nutrition, and time requirements. Minor issues such as typos and extra ingredients are present but do not significantly impact quality.
Test Case 4: Consistency Check	Ingredient, Cooking Time, Cuisine, Category	Cocoa Powder, Sugar, Eggs, 50 min, American, Dessert	3-Ingredient Chocolate Fudge Bites	The system generates a consistent and appropriate recipe that fits within the time constraint. This indicates improved reliability when constraints are less restrictive.

Test Case 5: Health-Focused Scenario	Ingredient, Nutrition Goal, Category	Cucumber, Olives, Feta, Low Fat, Salad	Quick Mediterranean Cucumber and Feta Salad	The output matches the category and ingredients but does not satisfy the low-fat requirement. This shows limitations in correctly interpreting and applying nutritional constraints.

Overall, the evaluation of the five test cases shows that the Budget Bites system performs well in generating coherent, realistic, and contextually appropriate recipes, particularly when ingredients, cuisine, and category constraints are clearly defined. In most cases, the system successfully incorporates required ingredients and produces structured cooking steps with reasonable nutritional estimates. However, performance is inconsistent in enforcing strict constraints, especially cooking time and nutrition goals. While some test cases meet time limits accurately, others significantly exceed them, indicating weak constraint control. Minimal-input scenarios also reveal limited inference capability, leading to less optimal outputs. Overall, the system demonstrates strong generative and contextual abilities but only moderate reliability as a constraint-driven recommendation system due to inconsistent adherence to user-specified limits.

4.5 System Demonstration (RAG-Based Chatbot)

To illustrate the end-to-end functionality of the Budget Bites system, we provide a demonstration video of the chatbot using the full pipeline (Retrieval-Augmented Generation). The video showcases how user inputs are processed, how relevant recipes are retrieved using FAISS and SentenceTransformers, and how the LLM generates the final recipe output.
Demo Video: https://www.youtube.com/shorts/oAw4VKbAHMc

5 Evaluation

5.1 Baselines

We compared two systems:

1. Baseline (LLM-only): The LLM generates a recipe using only the user input, without any retrieval.
2. Full System (Retrieval + LLM): Relevant recipes are first retrieved using FAISS, and then provided as context to the LLM to guide generation.

5.2 Evaluation Metrics

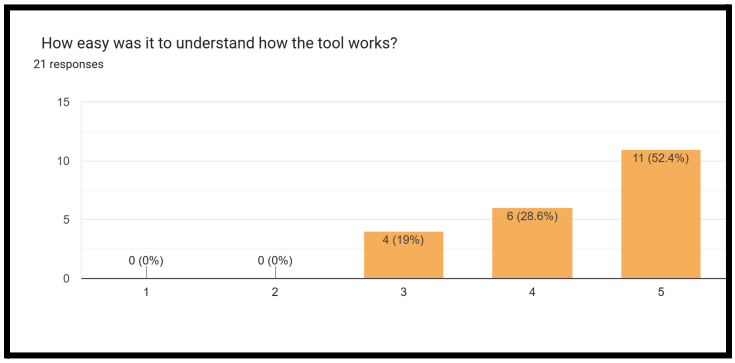
To evaluate the performance of the proposed system, we use a set of qualitative metrics designed to assess both the correctness and practical usefulness of generated recipes. Since the task involves open-ended text generation, evaluation focuses on alignment with user constraints and overall recipe quality.

1. Firstly, we measure whether the generated recipe correctly incorporates all or most of the user-provided ingredients to ensure that the output remains grounded in the input specification.
2. Secondly, we evaluate whether the generated recipe respects the maximum cooking time constraint provided by the user. Recipes that exceed the given time are considered less optimal.
3. Thirdly, we assess whether the output aligns with the selected cuisine type and recipe category, such as breakfast, snack, dinner, ensuring contextual consistency.
4. Fourthly, we check whether the recipe reflects the requested nutritional objective, such as high protein, low fat, low calorie, particularly in ingredient selection and cooking method.
5. Fifthly, we evaluate whether the recipe is logically structured and easy to follow. This includes clarity of steps and feasibility of preparation.

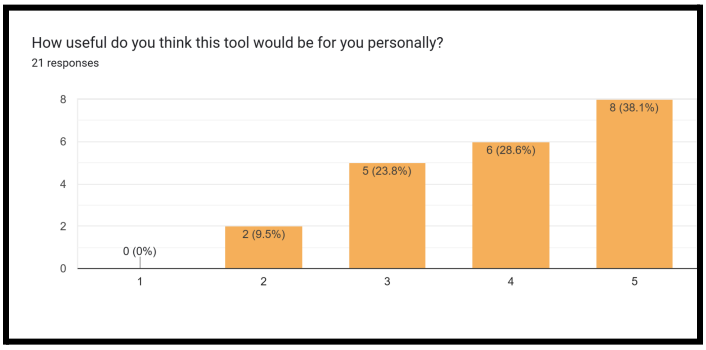
6. Finally we provide a holistic judgment of the recipe's quality, considering how helpful, relevant and complete the final recommendation is for the end user.

5.3 Market Survey

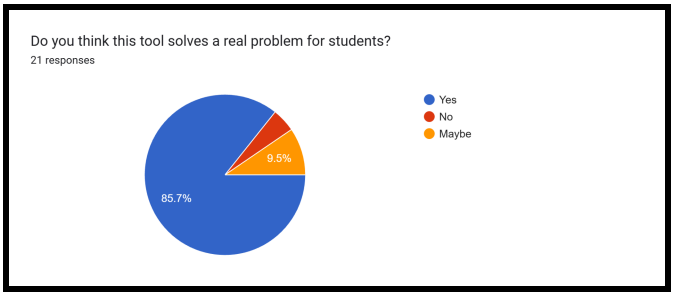
The user survey results indicate strong overall acceptance of the Budget Bites tool, particularly in terms of usability, usefulness and relevance to real-world cooking needs. Most participants currently rely on platforms such as YouTube, Google, TikTok and social media for recipe discovery, often reporting difficulties in deciding what to cook based on available ingredients. In contrast, the proposed tool was positively received for its ability to generate recipes from user-provided ingredients, with additional value added through nutrition-based filtering and cooking time constraints.



Across responses, users consistently rated the system as easy to understand (majority scoring 4–5), visually appealing, and useful for everyday situations such as meal planning during busy schedules, using leftover ingredients or deciding meals during exam periods. The most frequently highlighted valuable features were ingredient-based recipe generation, time-saving in meal planning and nutrition/macronutrient awareness. Many users also emphasized that current platforms lack structured nutritional information, which this system addresses effectively.
However, several recurring improvement suggestions were identified.



Overall, the survey results suggest that the tool addresses a genuine gap in existing recipe platforms by combining ingredient-based generation with nutritional and time constraints. While user satisfaction is generally high, incorporating stronger visual elements and extended personalization features would significantly improve adoption and usability.



5.4 Evaluation Result

Test Case	Scenario	Baseline LLM Output Issues	Full System (RAG) Improvements	Key Insight
Test Case 1: Time and Conflicting Nutrition	Sugar, egg, 15 min, Chinese, High Protein, Breakfast	Generates a quick recipe but contradicts high-protein goals (sugar-heavy, dessert-like). Poor constraint prioritization.	RAG generates a simpler egg-based dish, slightly reduces sugar usage, and aligns better with breakfast context. Still not fully high-protein.	Both struggle with conflicting constraints but RAG handles trade-offs better.
Test Case 2: Cuisine Constraint	Onion, Tomatoes, 45 min, Indian, High Protein	Generates generic curry-like dishes but fails high-protein requirements (mostly vegetables). Sometimes ignores cuisine authenticity or adds unrelated ingredients.	Produces a more Indian-aligned dish (e.g., curry format) and attempts to increase protein (e.g., adding lentils/paneer). Better ingredient coherence.	RAG improves contextual relevance but still imperfect in strict nutrition enforcement.
Test Case 3: Nutrition (Low-Calorie)	Vegetables, soy sauce, beef, low calorie, dinner	Baseline often generates high-calorie recipes (e.g., stir-fry with oil, large beef portions). Ignore calorie constraint entirely.	RAG produces lighter preparation methods (e.g., less oil, smaller portions, steaming/grilling). Better alignment with “low calorie.”	Baseline ignores nutrition; RAG partially enforces dietary constraints.
Test Case 4: Irregular Ingredients	Octopus, Seaweed, Vanilla, Flour, 60 min, Indian, Low Fat, Snack	Baseline produces unrealistic or incoherent recipes (random combination, poor culinary logic).	RAG generates more structured attempt, sometimes reinterprets ingredients into a plausible fusion dish. Still somewhat unnatural but improved coherence.	RAG improves reasoning with unusual inputs, but domain limits remain.

In summary, the integration of retrieval enhances the system's robustness, consistency and constraint-awareness, though further improvements are required for precise reasoning under conflicting conditions.

5.5 Demonstration of Baseline vs RAG Comparison

To further validate the evaluation results, we provide a comparison video demonstrating the differences between the baseline LLM and the full RAG-based system across multiple test cases. The video highlights how the baseline model often fails to satisfy key constraints such as nutrition goals and cooking time, while the RAG-based system produces more contextually relevant and constraint-aware outputs.
Comparison Video: https://youtube.com/shorts/EvygYpkIBEU?si=jBmeVP2skfZ7u_Hw

5.6 Analysis

The experimental results highlight clear differences in behavior between the baseline LLM and the full system integrating Retrieval-Augmented Generation (RAG). While both systems are capable of generating fluent and structurally correct recipes, their ability to satisfy user-defined constraints varies significantly.

The baseline LLM demonstrates strong generative capability but lacks constraint-awareness. Across multiple test cases, it frequently ignores or only partially satisfies critical requirements such as nutritional goals, cooking time and cuisine specificity. For example, in scenarios involving low-calorie or high-protein constraints, the baseline model tends to default to common recipe patterns without adjusting ingredient composition or cooking methods accordingly. Similarly, when presented with unusual or conflicting inputs, the model often produces incoherent or unrealistic outputs, indicating limited reasoning ability beyond surface-level text generation.

In contrast, the full system with RAG shows consistent improvements in contextual relevance and constraint satisfaction. By retrieving semantically similar recipes from the dataset and providing them as grounding context, the system is better able to align generated outputs with user intent. This is particularly evident in cuisine-specific scenarios, where the RAG system produces more authentic and structured recipes, and in nutrition-focused cases, where it adopts more appropriate cooking techniques such as reducing oil or adjusting ingredient proportions.

However, the analysis also reveals important limitations. Both systems struggle when handling conflicting constraints, such as combining low-protein ingredients with a high-protein requirement. In these cases, the RAG system performs better than the baseline by attempting partial adjustments but still fails to fully resolve the conflict. This suggests that while retrieval improves contextual grounding, it does not inherently provide logical reasoning or constraint prioritization.

Another key observation is that system performance improves as the number and clarity of constraints increase. When inputs are well-defined, both systems, especially the RAG-enhanced version, produce more accurate and relevant outputs. Conversely, minimal-input scenarios expose weaknesses in inference capability, where the model must make assumptions, often leading to suboptimal or inconsistent results.

Finally, the experiments demonstrate that RAG significantly enhances robustness in handling complex and irregular inputs. In cases involving uncommon ingredient combinations, the full system generates more coherent and structured recipes, whereas the baseline model often produces unrealistic or poorly connected outputs. This indicates that retrieval plays an important role in guiding the model toward plausible solutions, even when inputs deviate from typical patterns.

In summary, the integration of retrieval substantially improves the system's ability to generate contextually appropriate and constraint-aware recipes. Nevertheless, the system still lacks precise control over constraint enforcement, particularly for nutritional and conflicting requirements. Future improvements could focus on incorporating explicit constraint reasoning mechanisms or rule-based validation layers to further enhance reliability and accuracy.

6 Conclusion

6.1 Main Findings

This project presented **Budget Bites**, an AI-powered recipe recommendation system that combines a cleaned recipe dataset, FAISS-based retrieval, and Gemini 2.5 Flash to generate structured recipe suggestions. The system allows users to enter available ingredients, maximum cooking time, cuisine preference, nutrition goal, and recipe category, then returns a recipe name, description, ingredients, steps, total time, calories, protein, fats, and carbohydrates.

Overall, the system showed that retrieval-augmented generation can be useful for recipe recommendation because it grounds the LLM output in relevant recipes from the dataset instead of relying only on general LLM knowledge. Based on the experimental observations, Budget Bites performed best when users provided clear constraints such as ingredients, cuisine, category, and cooking time. The generated outputs were generally coherent, realistic, and easy to follow. However, the results also showed that the system was less reliable when inputs were minimal or when strict nutrition constraints had to be satisfied. The existing evaluation section notes that the system produced strong contextual outputs but only moderate reliability as a constraint-driven recommendation system because it sometimes failed to fully enforce cooking time or nutrition goals.

6.2 Contributions

The main contribution of this project is the development of a complete end-to-end LLM application for student-focused meal planning. The project goes beyond a basic recipe generator by combining several components: dataset cleaning, recipe enrichment, text embedding, FAISS retrieval, retrieval-augmented prompting, baseline comparison, and a Streamlit user interface.

Another contribution is the creation of a cleaned and enriched recipe dataset for retrieval and generation. The original dataset was incomplete, so important fields such as description, recipe steps, ingredients, and nutrition information were completed and standardized. This made the dataset more suitable for both semantic search and final recipe display.

The project also contributes a practical evaluation setup. The system was tested using multiple realistic scenarios, including limited ingredients, short cooking time, complete user constraints, health-focused goals, and consistency checks. In addition, the project compared the full retrieval-augmented system against an LLM-only baseline, which helped show the role of retrieval in improving grounded recipe generation. The report also includes survey-based feedback, where users generally found the tool useful, easy to understand, and relevant to real-world cooking needs.

6.3 Limitations

Despite these contributions, Budget Bites has several limitations. First, the system depends heavily on the size and quality of the recipe dataset. Since the dataset is limited, the system may not cover all cuisines, ingredients, dietary needs, or cooking styles. Expanding the dataset would improve the diversity and usefulness of the recommendations.

Second, the system does not always perfectly satisfy user constraints. Some outputs may exceed the requested cooking time, and nutrition goals such as low fat or high protein may not always be followed accurately. This limitation is especially important because the system is intended to support practical meal planning, where time and nutrition constraints matter.

Third, the nutrition values should be treated as approximate rather than medically verified. The system is not connected to a professional nutrition database, so it should not be used as a substitute for medical or dietitian advice.

Finally, the current implementation uses the free version of Gemini 2.5 Flash. Because of this, the system may face limits in speed, reliability, quota availability, and response quality. A more advanced API setup or stronger model could improve performance, reduce latency, and make the app more stable for regular use.

6.4 Future Work

Future improvements could focus on expanding the dataset, improving personalization, and making the interface more useful for real users. One important direction is to expand the recipe dataset to include more cuisines and more diverse food cultures. This would make the system more inclusive and improve its ability to handle different user preferences.

Another major improvement is to add dietary restriction filters. Future versions could support halal, vegan, vegetarian, gluten-free, dairy-free, nut-free, and allergy-aware recipe generation. This would make the system safer and more useful for users with specific dietary needs.

Based on the market survey and peer reviews, the most common request was for stronger visual features. Users suggested adding images of final dishes and step-by-step visual instructions. These additions would make the app easier to use, especially for students or beginner cooks who may need visual guidance. Users also suggested saved recipes, portion scaling, and links to videos or cooking demonstrations. These features would improve usability, learning support, and long-term engagement.

Overall, Budget Bites demonstrates the potential of combining retrieval and LLM generation for practical, personalized recipe recommendation. With a larger dataset, stronger constraint handling, dietary filters, visual guidance, and a more advanced API setup, the system could become a more reliable and complete meal-planning assistant.

7 Individual Contribution Statement

Report Section	**Samara Pires**	**Saung Hnin Phyu**	**Anna Tam Ly**
1 Introduction	33.33%	33.33%	33.33%
2 Literature Review	33.33%	33.33%	33.33%
3 Methodology	33.33%	33.33%	33.33%
4 Experiment	33.33%	33.33%	33.33%
5 Evaluation	33.33%	33.33%	33.33%
6 Conclusion	33.33%	33.33%	33.33%
Final Editing and Formatting	33.33%	33.33%	33.33%

Each teammate contributed equally to all sections. Percentages are shown as 33.33% due to rounding of one-third contributions.

8 Appendix

Github Link: https://github.com/Saung210/budgetbites_recipe_chatbot
System Demonstration Video: https://www.youtube.com/shorts/oAw4VKbAHM
Baseline vs RAG Comparison: https://www.youtube.com/shorts/oAw4VKbAHM

9 References

[1] Patil S, Kulkarni A, et al. Ingredient-based recipe recommendation system. Stanford University CS224W Project Report; 2019. Available from: https://snap.stanford.edu/class/cs224w-2019/project/26410425.pdf

[2] OpenAI. Prompt engineering guide [Internet]. San Francisco: OpenAI; [cited 2026 Apr 24]. Available from: https://platform.openai.com/docs/guides/prompt-engineering

[3] Lewis P, Perez E, Piktus A, Petroni F, Karpukhin V, Goyal N, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks [Internet]. arXiv; 2020 [cited 2026 Apr 24]. Available from: https://arxiv.org/abs/2005.11401

[4] Reimers N, Gurevych I. Sentence-BERT: Sentence embeddings using Siamese BERT-networks [Internet]. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP); 2019 [cited 2026 Apr 24]. Available from: https://aclanthology.org/D19-1410/

[5] Johnson J, Douze M, Jégou H. Billion-scale similarity search with GPUs [Internet]. arXiv; 2017 [cited 2026 Apr 24]. Available from: https://arxiv.org/abs/1702.08734

[6] Streamlit Inc. Streamlit documentation [Internet]. San Francisco: Streamlit Inc.; [cited 2026 Apr 24]. Available from: https://docs.streamlit.io/